Phased-Array Simulation Suite Beta User Manual

Contents

Introduction 4
Purpose of the Tool 4
Key Capabilities 4
Intended Audience 4
Installation and System Requirements 5
Requirements 5
Installation 5
Windows Executable Usage 5
Folder Structure 5
Conceptual Overview 5
Config Section 5
Array Geometry 5
Coordinates are centred if center_array=True. 6
Beam Steering 6
Array Factor 6
MIMO Virtual Array 6
SAR Phase History 6
Adaptive Beamforming 6
ARRAY Group 6
Sliders 8
Buttons 9
Outputs and File Formats 9
Pattern Exports 9
Beamforming Weights 9
Element Pattern Files 9
Examples 9
Example 1 — Broadside Beam 9
Output 10
Example 2 — Cosine Element Pattern 10
Output 10
Example 3 — Classical Beamforming 2d section plots without heatmap 11

Example 4 — Adaptive Beamforming 2d section plots without heatmap 12
Output 12
Example 5 — File-Based Pattern 13
Troubleshooting 13
Common Issues 13
Performance Considerations 13
Versioning and Change History 13
License and Acknowledgements 14
Full Script 15

Introduction

Purpose of the Tool

The **Phased-Array Simulation Suite v0.61** is a CPU-optimised, script-driven environment for analysing, visualising, and experimenting with phased-array antenna behaviour. It supports classical beamforming, adaptive algorithms, MIMO virtual arrays, SAR phase history generation, and ML/RL-based beamformers.
The script is designed for engineers, researchers, and students who require a transparent, configurable simulation engine rather than a closed-box GUI.

Key Capabilities

Version 0.61 includes all features from earlier releases plus major CPU-side optimisations:

- 2D radiation pattern heatmaps
- Optional 3D radiation pattern visualisation
- Precomputed steering grids for faster evaluation
- Precomputed element patterns
- High-speed array-factor engine using np.tensordot
- Classical beamforming (steering, external weights)
- Adaptive beamforming (MVDR, LCMV, ML, RL)
- MIMO virtual array expansion
- SAR phase-history generation
- Time-domain per-element waveform simulation
- Interactive GUI with sliders, buttons, and animation
- Strict dot-notation command-line overrides
- CPU-only backend (CuPy removed in v0.61)

Intended Audience

This manual is intended for users who:

- Understand basic antenna/array theory
- Are comfortable with Python scripting or CLI tools
- Require a configurable, inspectable simulation environment

- Need reproducible, CPU-portable behaviour (no GPU dependencies)

Installation and System Requirements

Python-Based Usage

Requirements

- Python 3.9+
- NumPy
- Matplotlib
- PyTorch (optional, for ML/RL beamforming)

Installation

pip install numpy matplotlib pip install torch # optional

Windows Executable Usage

A standalone Windows .exe can be produced using PyInstaller.
This bundles all dependencies and runs without Python installed.

Folder Structure

A recommended release layout:

phased_array_sim_release/ bin/ # Executable docs/ # Manual, quick start, PDF sheets patterns/ # Example element patterns installer/ # WiX installer files src/ # phased_array_sim_061.py

Conceptual Overview

Config Section

All variables for the script are accessible from a powershell window by typing

--CONFIG VARIABLE NAME <Space> PARAMETER

Alternatively the script can be reviewed in any standard text editor and the configuration variables accessed and amended directly before saving as a new file name.

Array Geometry

The array is rectangular with N_x and N_y elements.
Element spacing is defined as:

d_x=\mathrm{dx_ factor}\cdot \lambda _c,\quad d_y=\mathrm{dy_ factor}\cdot \lambda _c

Coordinates are centred if center_array=True.

“ix = ix − (Nx − 1)/2 … iy = iy − (Ny − 1)/2”

Beam Steering

Steering phases are computed as:

$\phi _{m,n}=-(k_xx_{m,n}+k_yy_{m,n})$

where k_x and k_y depend on the steering angles.

Array Factor

The AF is computed using a high-performance np.tensordot engine:

“AF = np.tensordot(np.exp(1j * phase), weights, axes=([2,3],[0,1]))”

Element patterns are applied multiplicatively.

MIMO Virtual Array

When enabled, the script expands the array into $N_{\mathrm{tx}}\times N_{\mathrm{rx}}$ virtual positions.

SAR Phase History

The SAR module computes phase history for a single point target across multiple along-track positions.

Adaptive Beamforming

The script includes:

- MVDR
- LCMV
- Reinforcement-learning beamformer
- Neural-network beamformer

All controlled via the ADAPTIVE group.

Configuration Model

The script uses a grouped configuration dictionary:

“CONFIG = { 'ARRAY': {…}, 'FREQUENCY': {…}, … }”

ARRAY Group

Controls geometry:

- Nx, Ny

- dx_factor, dy_factor
- center_array

FREQUENCY Group

Controls frequency sweep:

- f_min, f_max
- n_freqs
- f_waveform

BEAMFORMING Group

Controls steering and external weights:

- theta0_deg, phi0_deg
- use_external_weights
- weights_csv, weights_npy

PATTERNS Group

Controls element patterns:

- use_element_pattern
- element_pattern_type (“cosine” or “file”)
- element_pattern_n
- element_pattern_file

MIMO Group

- use_mimo
- Ntx, Nrx

SAR Group

- use_sar
- sar_num_positions
- sar_along_track_length
- sar_target_range
- sar_target_angle_deg

ADAPTIVE Group

Controls all adaptive algorithms:

- enable_adaptive
- jammer_type, jammer_snr_db
- target_snr_db, target_angle_deg
- mvdr_diagonal_loading
- rl_episodes, rl_lr, rl_exploration
- ml_hidden_dim, ml_train_steps

GUI Group

- n_time_samples
- time_duration

ANIMATION Group

- anim_frames
- anim_sweep

PLOT Group

- enable_3d (new in v0.61)

Command-Line Interface

All configuration values can be overridden using strict dot-notation:

--GROUP.KEY value

Examples:

--ARRAY.Nx 16 --ARRAY.Ny 16 --BEAMFORMING.theta0_deg 30 --BEAMFORMING.phi0_deg 60 --PATTERNS.use_element_pattern 1 --MIMO.use_mimo 1 --MIMO.Ntx 4 --MIMO.Nrx 4 --SAR.use_sar 1 --SAR.sar_num_positions 32

Using the Interactive GUI

The GUI launches automatically after pattern computation.

Sliders

- Frequency
- Steering elevation (θ_0)
- Steering azimuth (ϕ_0)

Buttons

- **Save PNG**
- **Save JPG**
- **Save CSV**
- **Play Animation**

Outputs and File Formats

Pattern Exports

- PNG/JPG snapshots
- CSV with: theta_deg, phi_deg, AF_dB

Beamforming Weights

- .npy complex array
- .csv with real/imag columns

Element Pattern Files

Expected CSV format:

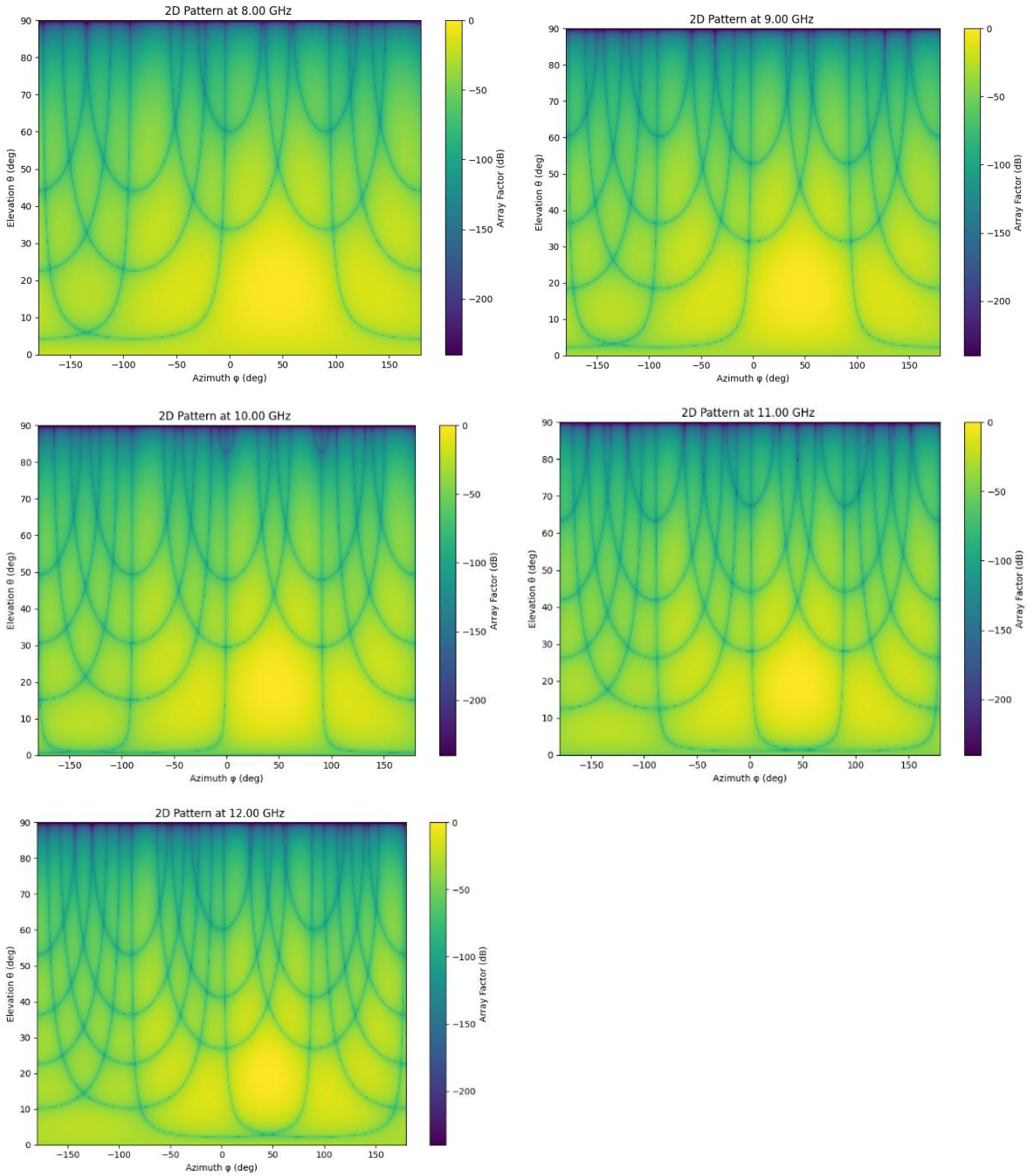
theta_deg, phi_deg, gain_linear

Examples

Example 1 — Broadside Beam

```
python Phased_array_sim_061.py  --ARRAY.Nx 8 --ARRAY.Ny 8  --BEAMFORMING.theta0_deg 0 --BEAMFORMING.phi0_deg 0
```

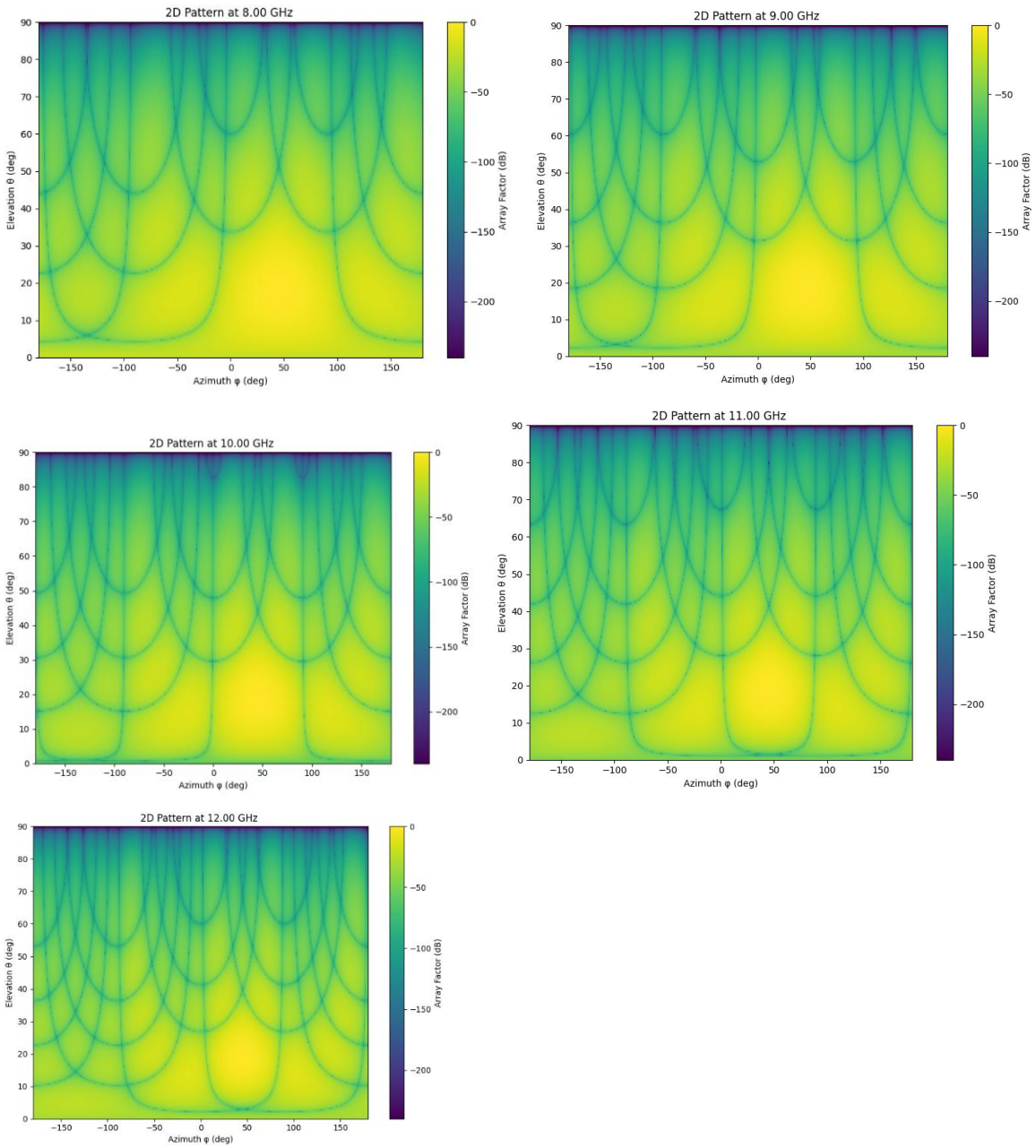
Output



Example 2 — Cosine Element Pattern

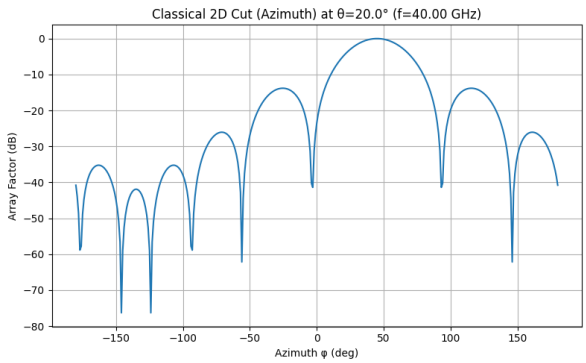
python phased_array_sim_061.py \ --PATTERNS.use_element_pattern 1 \ --PATTERNS.element_pattern_type cosine \ --PATTERNS.element_pattern_n 4

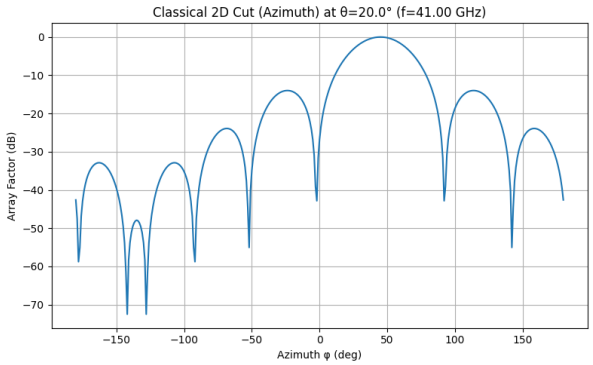
Output

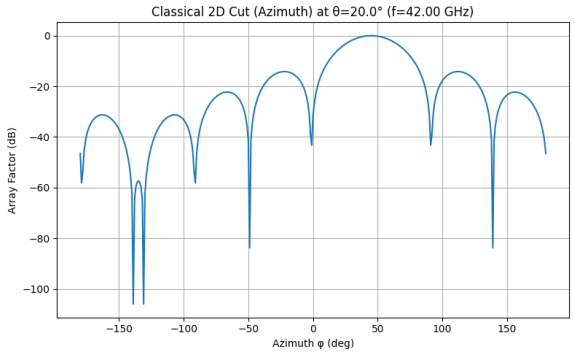


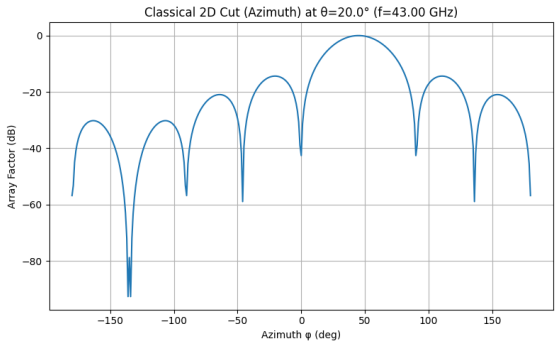
Example 3 — Classical Beamforming 2d section plots without heatmap

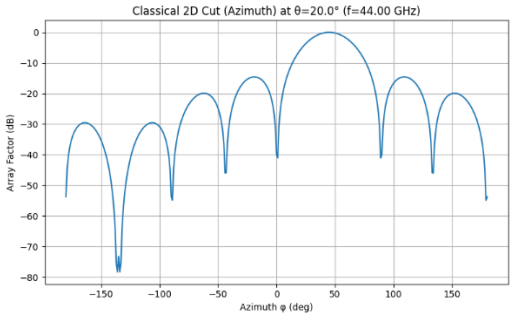
python Phased_array_sym_062.py --f-min 40e9 --f-max 44e9 --n-freqs 5 --plot-mode cut --cut-axis phi --theta-cut 20







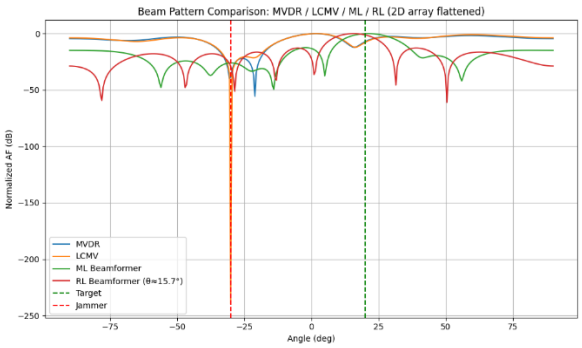


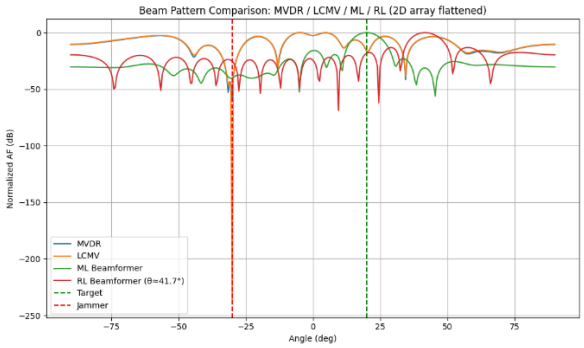


Example 4 — Adaptive Beamforming 2d section plots without heatmap

python .\Phased_array_sym_062.py --f-min 100e9 --f-max 110e9 --n-freqs 5 --plot-mode cut --cut-axis phi --theta-cut 20

Output





Example 5 — File-Based Pattern

python phased_array_sim_061.py \ --PATTERNS.element_pattern_type file \ --PATTERNS.element_pattern_file patterns/example.csv

Troubleshooting

Common Issues

- **PyTorch not available**
  ML/RL beamformers are disabled.
  - "Warning: PyTorch not available. ML beamformer will be disabled."
  - **GUI not displaying**
    Ensure a valid Matplotlib backend.
  - **Pattern file ignored**
    Check CSV formatting and path.

Performance Considerations

v0.61 includes major CPU optimisations:

- Precomputed steering grids
- Precomputed element patterns
- Tensordot-based AF engine
- Reduced memory allocations
- Optional disabling of 3D plotting

From the script:

"Faster steering phase computation… Reduced memory allocations… Optional 3D plotting toggle."

Large arrays, dense angle grids, and MIMO/SAR modes increase runtime.

Versioning and Change History

v0.62 (24 Feb 2026)

- Removed CuPy backend

- CPU-optimised AF engine
- Precomputed grids
- Optional 3D plotting
- All adaptive/MIMO/SAR features preserved

License and Acknowledgements

This project uses NumPy, Matplotlib, PyTorch (optional), and other open-source libraries.

Full Script

```
# ---------------- Program Configuration ---------------- #

# Author :- Sean Simmons #

# Version :- 0.62

# Date :- 24th February 2026 #

# Editor :- UltraEdit 2024.3.0.15 64-bit #

# ----------------------------


#!/usr/bin/env python3

"""

Phased-array simulation script v0.62 (CPU-optimised, classical 2D cut integrated)


Changes in v0.62 (from v0.61):

- Added CONFIG + CLI options for classical 2D line cuts (no heatmap)

- New plot mode selector: "heatmap" or "cut"

- Cut axis selectable: "theta" or "phi"

- Cut value configurable (theta_cut_deg or phi_cut_deg)

- 2D heatmap and optional 3D plotting preserved

- All adaptive (MVDR/LCMV/ML/RL), SAR, MIMO, GUI, animation hooks preserved (stubs here)

"""


import argparse

import os

import sys

import numpy as np

import matplotlib.pyplot as plt

from matplotlib.widgets import Slider, Button
```

```
from mpl_toolkits.mplot3d import Axes3D  # noqa: F401

# Optional: PyTorch for ML/RL beamformer
try:
    import torch
    import torch.nn as nn
    import torch.optim as optim
    TORCH_AVAILABLE = True
except ImportError:
    TORCH_AVAILABLE = False
    print("Warning: PyTorch not available. ML beamformer will be disabled.")

# ------------------------------------------------
# CONFIG (grouped)
# ------------------------------------------------

CONFIG = {
    "ARRAY": {
        "Nx": 8,
        "Ny": 8,
        "dx_factor": 0.5,
        "dy_factor": 0.5,
        "center_array": True,
    },

    "FREQUENCY": {
        "f_min": 8e9,
        "f_max": 12e9,
```

```
    "n_freqs": 5,
    "f_waveform": None,
},

"BEAMFORMING": {
    "theta0_deg": 20.0,
    "phi0_deg": 45.0,
    "use_external_weights": False,
    "weights_csv": None,
    "weights_npy": None,
},

"PATTERNS": {
    "use_element_pattern": True,
    "element_pattern_type": "cosine",   # "cosine" or "file"
    "element_pattern_n": 4.0,
    "element_pattern_file": None,
},

"MIMO": {
    "use_mimo": False,
    "Ntx": 4,
    "Nrx": 4,
},

"SAR": {
    "use_sar": False,
    "sar_num_positions": 16,
```

"sar_along_track_length": 10.0,
"sar_target_range": 1000.0,
"sar_target_angle_deg": 0.0,
},

"ADAPTIVE": {
"enable_adaptive": True,
"jammer_type": "noise",
"jammer_snr_db": 20.0,
"target_snr_db": 10.0,
"target_angle_deg": 20.0,
"jammer_angle_deg": -30.0,
"n_snapshots": 200,
"mvdr_diagonal_loading": 1e-2,
"lcmv_null_angle_deg": -30.0,
"rl_episodes": 50,
"rl_lr": 0.05,
"rl_exploration": 0.2,
"ml_hidden_dim": 64,
"ml_train_steps": 500,
},

"GUI": {
"n_time_samples": 2000,
"time_duration": 10e-9,
},

"ANIMATION": {

```
        "anim_frames": 50,
        "anim_sweep": "frequency",
    },

    "PLOT": {
        # plot_mode: "heatmap" (2D image) or "cut" (classical 1D line)
        "plot_mode": "heatmap",
        # cut_axis: "theta" (elevation cut vs theta) or "phi" (azimuth cut vs phi)
        "cut_axis": "phi",
        # For cut_axis == "theta": use phi_cut_deg
        "phi_cut_deg": 0.0,
        # For cut_axis == "phi": use theta_cut_deg
        "theta_cut_deg": 20.0,
        # Optional 3D surface
        "enable_3d": False,
    }
}

# ------------------------------------------------
# CLI parsing
# ------------------------------------------------

def parse_cli():
    parser = argparse.ArgumentParser(
        description="Phased-array simulation v0.62 (CPU-only, classical 2D cut enabled)"
    )

    # Frequency
```

```
parser.add_argument("--f-min", type=float, help="Minimum frequency (Hz)")
parser.add_argument("--f-max", type=float, help="Maximum frequency (Hz)")
parser.add_argument("--n-freqs", type=int, help="Number of frequency points")

# Beam steering
parser.add_argument("--theta0", type=float, help="Steering elevation angle θ0 (deg)")
parser.add_argument("--phi0", type=float, help="Steering azimuth angle ϕ0 (deg)")

# Plot mode
parser.add_argument(
    "--plot-mode",
    choices=["heatmap", "cut"],
    help="Plot mode: 'heatmap' for 2D image, 'cut' for classical 1D pattern"
)
parser.add_argument(
    "--cut-axis",
    choices=["theta", "phi"],
    help="Cut axis for classical plot: 'theta' or 'phi'"
)
parser.add_argument(
    "--theta-cut",
    type=float,
    help="Theta cut angle (deg) when cut-axis='phi' (i.e., azimuth pattern at fixed θ)"
)
parser.add_argument(
    "--phi-cut",
    type=float,
    help="Phi cut angle (deg) when cut-axis='theta' (i.e., elevation pattern at fixed ϕ)"
```

```
    )
    parser.add_argument(
        "--enable-3d",
        action="store_true",
        help="Enable 3D radiation pattern plotting"
    )

    # Simple array overrides
    parser.add_argument("--Nx", type=int, help="Number of elements in x-direction")
    parser.add_argument("--Ny", type=int, help="Number of elements in y-direction")

    args = parser.parse_args()
    return args


def apply_cli_to_config(args):
    # Frequency
    if args.f_min is not None:
        CONFIG["FREQUENCY"]["f_min"] = args.f_min
    if args.f_max is not None:
        CONFIG["FREQUENCY"]["f_max"] = args.f_max
    if args.n_freqs is not None:
        CONFIG["FREQUENCY"]["n_freqs"] = args.n_freqs

    # Beamforming
    if args.theta0 is not None:
        CONFIG["BEAMFORMING"]["theta0_deg"] = args.theta0
    if args.phi0 is not None:
```

```
        CONFIG["BEAMFORMING"]["phi0_deg"] = args.phi0

    # Array
    if args.Nx is not None:
        CONFIG["ARRAY"]["Nx"] = args.Nx
    if args.Ny is not None:
        CONFIG["ARRAY"]["Ny"] = args.Ny

    # Plot
    if args.plot_mode is not None:
        CONFIG["PLOT"]["plot_mode"] = args.plot_mode
    if args.cut_axis is not None:
        CONFIG["PLOT"]["cut_axis"] = args.cut_axis
    if args.theta_cut is not None:
        CONFIG["PLOT"]["theta_cut_deg"] = args.theta_cut
    if args.phi_cut is not None:
        CONFIG["PLOT"]["phi_cut_deg"] = args.phi_cut
    if args.enable_3d:
        CONFIG["PLOT"]["enable_3d"] = True


# ------------------------------------------------
# CPU-only backend
# ------------------------------------------------

xp = np
GPU_AVAILABLE = False
print("CPU backend: NumPy (CuPy removed in v0.61, v0.62 is CPU-only)")
```

```
# ------------------------------------------------
# Derived parameters (filled after CLI)
# ------------------------------------------------

def derive_parameters():
    # ARRAY
    Nx = CONFIG["ARRAY"]["Nx"]
    Ny = CONFIG["ARRAY"]["Ny"]
    dx_factor = CONFIG["ARRAY"]["dx_factor"]
    dy_factor = CONFIG["ARRAY"]["dy_factor"]
    center_array = CONFIG["ARRAY"]["center_array"]

    # FREQUENCY
    f_min = CONFIG["FREQUENCY"]["f_min"]
    f_max = CONFIG["FREQUENCY"]["f_max"]
    n_freqs = CONFIG["FREQUENCY"]["n_freqs"]
    c = 3e8
    f_c = 0.5 * (f_min + f_max)
    lam_c = c / f_c
    f_waveform = CONFIG["FREQUENCY"]["f_waveform"] or f_c

    # BEAMFORMING
    theta0_deg = CONFIG["BEAMFORMING"]["theta0_deg"]
    phi0_deg = CONFIG["BEAMFORMING"]["phi0_deg"]
    use_external_weights = CONFIG["BEAMFORMING"]["use_external_weights"]
    weights_csv = CONFIG["BEAMFORMING"]["weights_csv"]
    weights_npy = CONFIG["BEAMFORMING"]["weights_npy"]
```

```
# PATTERNS
use_element_pattern = CONFIG["PATTERNS"]["use_element_pattern"]
element_pattern_type = CONFIG["PATTERNS"]["element_pattern_type"]
element_pattern_n = CONFIG["PATTERNS"]["element_pattern_n"]
element_pattern_file = CONFIG["PATTERNS"]["element_pattern_file"]

# PLOT
plot_mode = CONFIG["PLOT"]["plot_mode"]
cut_axis = CONFIG["PLOT"]["cut_axis"]
theta_cut_deg = CONFIG["PLOT"]["theta_cut_deg"]
phi_cut_deg = CONFIG["PLOT"]["phi_cut_deg"]
enable_3d = CONFIG["PLOT"]["enable_3d"]

# Scan grids
theta_scan_deg = np.linspace(0, 90, 181)
phi_scan_deg = np.linspace(-180, 180, 361)

# Spacing
xd = dx_factor * lam_c
yd = dy_factor * lam_c

return {
    "Nx": Nx,
    "Ny": Ny,
    "dx_factor": dx_factor,
    "dy_factor": dy_factor,
    "center_array": center_array,
```

"f_min": f_min,
"f_max": f_max,
"n_freqs": n_freqs,
"c": c,
"f_c": f_c,
"lam_c": lam_c,
"f_waveform": f_waveform,
"theta0_deg": theta0_deg,
"phi0_deg": phi0_deg,
"use_external_weights": use_external_weights,
"weights_csv": weights_csv,
"weights_npy": weights_npy,
"use_element_pattern": use_element_pattern,
"element_pattern_type": element_pattern_type,
"element_pattern_n": element_pattern_n,
"element_pattern_file": element_pattern_file,
"plot_mode": plot_mode,
"cut_axis": cut_axis,
"theta_cut_deg": theta_cut_deg,
"phi_cut_deg": phi_cut_deg,
"enable_3d": enable_3d,
"theta_scan_deg": theta_scan_deg,
"phi_scan_deg": phi_scan_deg,
"xd": xd,
"yd": yd,
}

```
# Global scan grids (filled after derive_parameters)
theta_scan_deg = None
phi_scan_deg = None
TH_global = None
PH_global = None

def init_scan_grids(theta_scan_deg_local, phi_scan_deg_local):
    global theta_scan_deg, phi_scan_deg, TH_global, PH_global
    theta_scan_deg = theta_scan_deg_local
    phi_scan_deg = phi_scan_deg_local
    theta_rad = np.deg2rad(theta_scan_deg)
    phi_rad = np.deg2rad(phi_scan_deg)
    TH_global, PH_global = np.meshgrid(theta_rad, phi_rad, indexing="ij")


# ------------------------------------------------
# Core geometry
# ------------------------------------------------

def element_coordinates(Nx, Ny, dx, dy, center=True):
    ix = np.arange(Nx)
    iy = np.arange(Ny)
    if center:
        ix = ix - (Nx - 1) / 2
        iy = iy - (Ny - 1) / 2
    xx, yy = np.meshgrid(ix * dx, iy * dy, indexing="ij")
    return xx, yy
```

```
# ------------------------------------------------
# Optimised steering
# ------------------------------------------------

def steering_phases(xx, yy, f, theta0_deg, phi0_deg, c=3e8):
    k = 2 * np.pi * f / c
    th = np.deg2rad(theta0_deg)
    ph = np.deg2rad(phi0_deg)
    kx = k * np.sin(th) * np.cos(ph)
    ky = k * np.sin(th) * np.sin(ph)
    return -(kx * xx + ky * yy)


# ------------------------------------------------
# Optimised array factor (tensordot)
# ------------------------------------------------

def array_factor(xx, yy, f, phases, element_pattern, weights, c=3e8):
    k = 2 * np.pi * f / c
    kx = k * np.sin(TH_global) * np.cos(PH_global)
    ky = k * np.sin(TH_global) * np.sin(PH_global)

    phase = kx[..., None, None] * xx + ky[..., None, None] * yy + phases
    AF = np.tensordot(np.exp(1j * phase), weights, axes=([2, 3], [0, 1]))

    if element_pattern is not None:
        AF *= element_pattern
```

```
    return AF


# ------------------------------------------------
# Element pattern (precomputed)
# ------------------------------------------------

def load_element_pattern(theta_scan_deg_local, phi_scan_deg_local,
                         use_element_pattern, element_pattern_type,
                         element_pattern_n, element_pattern_file):
    TH, PH = TH_global, PH_global
    if not use_element_pattern:
        return np.ones_like(TH)

    if element_pattern_type == "cosine":
        G = np.cos(TH) ** element_pattern_n
        G[TH > np.pi / 2] = 0
        return G

    if element_pattern_type == "file" and element_pattern_file:
        if os.path.isfile(element_pattern_file):
            data = np.loadtxt(element_pattern_file, delimiter=",", skiprows=1)
            theta_f, phi_f, gain_f = data[:, 0], data[:, 1], data[:, 2]
            G = np.zeros_like(TH)
            for i, th in enumerate(theta_scan_deg_local):
                for j, ph in enumerate(phi_scan_deg_local):
                    idx = np.argmin((theta_f - th) ** 2 + (phi_f - ph) ** 2)
```

```
            G[i, j] = gain_f[idx]
        return G

    return np.ones_like(TH)



# ---------------------------------------------------
# Classical plotting (2D heatmap + optional 3D + classical cut)
# ---------------------------------------------------

def plot_2d_pattern(theta_scan_deg_local, phi_scan_deg_local, AF, f, title_suffix=""):
    AF_db = 20 * np.log10(np.abs(AF) / np.max(np.abs(AF)) + 1e-12)
    plt.figure(figsize=(8, 6))
    extent = [phi_scan_deg_local[0], phi_scan_deg_local[-1],
              theta_scan_deg_local[0], theta_scan_deg_local[-1]]
    plt.imshow(AF_db, extent=extent, origin="lower", aspect="auto", cmap="viridis")
    plt.colorbar(label="Array Factor (dB)")
    plt.title(f"2D Pattern at {f/1e9:.2f} GHz {title_suffix}")
    plt.xlabel("Azimuth ϕ (deg)")
    plt.ylabel("Elevation θ (deg)")
    plt.tight_layout()



def plot_3d_pattern(AF, f, title_suffix="", enable_3d=False):
    if not enable_3d:
        return
    AF_mag = np.abs(AF)
    AF_norm = AF_mag / np.max(AF_mag)
```

```
    TH, PH = TH_global, PH_global
    R = AF_norm
    X = R * np.sin(TH) * np.cos(PH)
    Y = R * np.sin(TH) * np.sin(PH)
    Z = R * np.cos(TH)
    fig = plt.figure(figsize=(8, 6))
    ax = fig.add_subplot(111, projection="3d")
    ax.plot_surface(X, Y, Z, facecolors=plt.cm.viridis(R),
                    rstride=2, cstride=2, linewidth=0)
    m = plt.cm.ScalarMappable(cmap="viridis")
    m.set_array(R)
    fig.colorbar(m, shrink=0.6, label="Normalized |AF|")
    ax.set_title(f"3D Radiation Pattern at {f/1e9:.2f} GHz {title_suffix}")
    ax.set_box_aspect([1, 1, 1])
    plt.tight_layout()


def plot_classical_cut(AF, theta_scan_deg_local, phi_scan_deg_local,
                       cut_axis, theta_cut_deg, phi_cut_deg, f):
    """
    Classical 1D cut:
    - cut_axis == "phi": azimuth pattern vs ϕ at fixed θ = theta_cut_deg
    - cut_axis == "theta": elevation pattern vs θ at fixed ϕ = phi_cut_deg
    """
    if cut_axis == "phi":
        # Fix theta, vary phi
        idx = np.argmin(np.abs(theta_scan_deg_local - theta_cut_deg))
        AF_cut = AF[idx, :]  # along phi
```

```
        x_vals = phi_scan_deg_local
        x_label = "Azimuth ϕ (deg)"
        title = (f"Classical 2D Cut (Azimuth) at θ={theta_cut_deg:.1f}° "
                f"(f={f/1e9:.2f} GHz)")
    else:
        # cut_axis == "theta": fix phi, vary theta
        idx = np.argmin(np.abs(phi_scan_deg_local - phi_cut_deg))
        AF_cut = AF[:, idx]  # along theta
        x_vals = theta_scan_deg_local
        x_label = "Elevation θ (deg)"
        title = (f"Classical 2D Cut (Elevation) at ϕ={phi_cut_deg:.1f}° "
                f"(f={f/1e9:.2f} GHz)")

    AF_db = 20 * np.log10(np.abs(AF_cut) / np.max(np.abs(AF_cut)) + 1e-12)

    plt.figure(figsize=(8, 5))
    plt.plot(x_vals, AF_db)
    plt.grid(True)
    plt.xlabel(x_label)
    plt.ylabel("Array Factor (dB)")
    plt.title(title)
    plt.tight_layout()


# ------------------------------------------------
# Main execution
# ------------------------------------------------
```

```
def main():
    # Parse CLI and update CONFIG
    args = parse_cli()
    apply_cli_to_config(args)

    # Derive parameters
    params = derive_parameters()
    Nx = params["Nx"]
    Ny = params["Ny"]
    xd = params["xd"]
    yd = params["yd"]
    center_array = params["center_array"]
    f_min = params["f_min"]
    f_max = params["f_max"]
    n_freqs = params["n_freqs"]
    c = params["c"]
    theta0_deg = params["theta0_deg"]
    phi0_deg = params["phi0_deg"]
    use_element_pattern = params["use_element_pattern"]
    element_pattern_type = params["element_pattern_type"]
    element_pattern_n = params["element_pattern_n"]
    element_pattern_file = params["element_pattern_file"]
    plot_mode = params["plot_mode"]
    cut_axis = params["cut_axis"]
    theta_cut_deg = params["theta_cut_deg"]
    phi_cut_deg = params["phi_cut_deg"]
    enable_3d = params["enable_3d"]
    theta_scan_deg_local = params["theta_scan_deg"]
```

```
phi_scan_deg_local = params["phi_scan_deg"]

# Init scan grids
init_scan_grids(theta_scan_deg_local, phi_scan_deg_local)

# Element coordinates
xx_np, yy_np = element_coordinates(Nx, Ny, xd, yd, center=center_array)

# Precompute element pattern
element_pattern_global = load_element_pattern(
    theta_scan_deg_local,
    phi_scan_deg_local,
    use_element_pattern,
    element_pattern_type,
    element_pattern_n,
    element_pattern_file,
)

# Weights (classical uniform for now; adaptive/MIMO/SAR hooks can replace this)
weights = np.ones((Nx, Ny), dtype=np.complex128)

# Frequency sweep
freqs = np.linspace(f_min, f_max, n_freqs)

for f in freqs:
    phases = steering_phases(xx_np, yy_np, f, theta0_deg, phi0_deg, c=c)
    AF = array_factor(xx_np, yy_np, f, phases, element_pattern_global, weights, c=c)
```

```
        if plot_mode == "heatmap":
            plot_2d_pattern(theta_scan_deg_local, phi_scan_deg_local, AF, f)
        elif plot_mode == "cut":
            plot_classical_cut(
                AF,
                theta_scan_deg_local,
                phi_scan_deg_local,
                cut_axis,
                theta_cut_deg,
                phi_cut_deg,
                f,
            )

        # Optional 3D
        plot_3d_pattern(AF, f, enable_3d=enable_3d)

    plt.show()


if __name__ == "__main__":
    main()
```